Secure Software Development — Project Report

Topic: Implementation and Evaluation of a Data Integrity Verification Service

Student Name: Haya Aly Bakry

Student ID :241004475

Development Environment: C++ | Microsoft Visual Studio

1. Introduction & Security Context

In secure software development, Data Integrity is one of the foundational pillars of the CIA Triad (Confidentiality, Integrity, Availability). It guarantees that data remains accurate, complete, and uncorrupted during storage or transmission, ensuring protection against unauthorized modifications.

To achieve integrity verification without storing full duplicate copies of files, cryptographically sound Hash Functions are utilized. A hash function processes arbitrarily sized input data and returns a fixed-length string (often called a message digest or fingerprint). Key properties of hashing functions include:

Determinism: The exact same input always produces the exact same hash output.

Pre-image Resistance: It is computationally infeasible to reverse-engineer the original data from its hash.

Avalanche Effect: A minor change in the input (even a single bit or character) results in a drastically different output hash, rendering unauthorized tampering immediately visible.

2. Technical Design & Implementation

The application was built as a standalone C++ Console Application within Microsoft Visual Studio. It operates directly on file bitstreams to ensure compatibility across all file formats (text, binary, or executable files).

Core Components of Secure7thproject.cpp: Binary Data Ingestion (readBinaryFile):

File streams (std::ifstream) are opened using binary flags (ios::binary | ios::ate) to read raw bytes directly from disk without text encoding distortions.

The stream offset determines exact file byte sizes before populating a dynamic byte buffer (std::vector).

Hash Computation (calculateSimpleHash):

The byte buffer is processed using an iterative hashing calculation based on bitwise operations and prime multipliers (5381 initializer with bit-shifts: hash=(hash×33)+b).

The resulting numeric value is formatted into a standardized 16-character hexadecimal representation (std::stringstream with std::hex and zero-padded width formatting).

Integrity Check Routine (main):

Automated file creation generates a standard reference file (test.txt) upon initialization if absent.

Upon execution, the current file contents are hashed dynamically, establishing an automated baseline comparison.

3. Experimental Testing & Demonstration

To evaluate the integrity verification mechanism, two primary operational states were tested and analyzed:

Baseline Execution (Untampered State) Input File Content: "Hello, this is a secure message for Data Integrity!"

Computed Output Hash: 000000009f98a3c7

Result: The integrity checker confirmed that the file was clean, authentic, and unmodified ([+] Status: Data Integrity Verified!).

Tampering Demonstration (Tampered State) Action Taken: The file test.txt was manually edited to alter a single character before saving.

Computed Output Hash: The newly calculated hexadecimal digest altered completely away from 000000009f98a3c7.

Result: The system detected the mismatch between expected and computed digests, effectively proving that data integrity had been compromised.

4. Conclusion & Declaration

This implementation effectively demonstrates how lightweight hash computation algorithms serve as effective integrity checks in secure software architecture.

Statement of Assistance & Tools In accordance with ECE4113 course ethics guidelines, foundational guidance and code structuring for this project were supported by generative AI tools (Gemini) and C++ standard library documentations. All source code, logic

execution, and theoretical principles have been thoroughly studied, tested, and verified to ensure complete personal understanding for the oral viva evaluation.